lifewood

# lifewood-branding

Developer Manual

Prepared by: Lifewood PH Team

Date: August 14, 2026

# Contents



## How to use this document

Audience: Developers joining the team who need to ship a change in their first week. Assumes working knowledge of the listed technologies but no familiarity with this specific codebase.

Conventions: monospace text = literal commands or code you type; ‹angle brackets› = replace with your value; callout boxes highlight warnings or prerequisites.

1.0

# Overview & Architecture

lifewood-branding — lifewood-branding is a Python application. See the sections below for setup, architecture, and development workflow details.

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | Python | User interface & client logic |
| Backend | TBD from repo scan | API & business logic |
| Database | TBD from repo scan | Data persistence |
| Build | | Build toolchain |
| Hosting | | Deployment target |

Architecture diagram: The system follows a standard client-server architecture. The frontend communicates with the backend via REST API or direct database queries (Supabase). Authentication flows through Supabase Auth. Static assets and API routes are served via the hosting provider's edge network.

2.0

# Prerequisites

Ensure the following tools are installed before setting up the project:

| Tool | Required Version | Verify With |
|---|---|---|
| Node.js | >=18 | node --version |
| npm / pnpm / yarn | Latest stable | npm --version |
| Git | >=2.30 | git --version |
| Supabase account | Any | supabase.com |

> **Note**
> If you use nvm (Node Version Manager), run nvm use to pick the correct Node version from .nvmrc if present.

3.0

# Project Setup

Follow these steps to get the project running locally for the first time:

| Step | Expected Outcome |
| --- | --- |
| Clone the repository: git clone <repo-url> && cd lifewood-branding | Verify directory exists locally |
| Install dependencies: npm install | node_modules/ created, no errors |
| Copy environment file: cp .env.example .env | .env file created |
| Fill in environment variables (see Section 6.0) | App connects to backend services |
| Start dev server: npm run dev | Browser opens at localhost:5173 or similar |

QUICK START (NPM)

```
npm install
npm run dev
```

If running the dev server for the first time, expect a short delay for dependency resolution and Vite/Rspack pre-bundling.

4.0

# Code Architecture

The project root contains 23 source files across 1 languages. Key directories:

DIRECTORY TREE

```
__pycache__/
assets/
  manrope_extracted/
build_admin_manual.py
build_developer_manual.py
build_lifedata_user_manual.py
build_lifedata_workflows.py
build_overview_deck.py
build_prmce_overview_deck.py
build_prmce_user_manual.py
build_prmce_workflows.py
build_storyboard_studio_user_manual.py
```

Architecture flow: entry point → routes → page components → shared child components; data flows API/database → service layer → component state → render.

5.0

# Component Reference

No components directory detected under src/. The application may use a flat file structure or page-based routing instead of a components/ folder.

For a complete component inventory, run: find lifewood-branding/src -name '*.tsx' -o -name '*.ts' | head -30

6.0

# Configuration Reference

The application is configured through environment variables defined in .env (development) or the hosting dashboard (production).

| Name | Type | Required | Example | Effect |
|---|---|---|---|---|
| VITE_SUPABASE_URL | string | Yes | https://your-project.supabase.co | Supabase project URL for database & auth |
| VITE_SUPABASE_ANON_KEY | string | Yes | eyJhbGciOiJIUzI1NiIs... | Public anon key for Supabase client |

**Warning**

Never commit .env files. Never expose service_role keys. Use the hosting dashboard for production environment variables.

7.0

# Data & Auth Flow

Data flows through a unidirectional pipeline: UI event → service/API call → response → state update → re-render.

1. Component mounts or user triggers an action.
2. Service layer makes the request (Supabase / fetch / axios).
3. Response returns as JSON or Supabase query result.
4. State is updated via a React setter or query cache.
5. React re-renders affected components with the new data.

Authentication flow:

1. User enters email/password on the login page.
2. Supabase Auth validates credentials, returns a session JWT.
3. JWT is stored in the browser and sent with subsequent API requests via the Authorization header.
4. Supabase RLS policies use the JWT's user ID for row filtering.
5. On logout, the JWT is cleared and the UI returns to login state.

8.0

# Extending the Platform

Common extension tasks follow a recipe pattern: goal → steps → files touched → verification.

Recipe: Add a new page

1. Create a new file in src/pages/ or app/ directory.
2. Define the page component and add its route to the router.
3. Add the navigation link in the sidebar or nav bar.
4. Verify: navigate to the new route in the browser.

Recipe: Add a new API endpoint

1. Create the serverless function or API route file.
2. Define the request handler with input validation.
3. Connect the frontend to the new endpoint.
4. Verify: call the endpoint and check the response.

8.1

# Modifying Components & Verification

Recipe: Modify an existing component

1. Locate the component file in src/components/.
2. Identify the props interface and state management.
3. Make the change following existing patterns.
4. Run npm run lint, then npm run build to verify.

Every extension ships with verification: run the lint and build commands, check the changed flow in the browser, and keep the change small enough for a clean review.

9.0

# Commands & Workflow

The following commands are defined in package.json scripts:

| Command | Action | When to Run |
| --- | --- | --- |
| npm run dev | Start dev server | Development |
| npm run build | Production build | Pre-deploy |
| npm run lint | Lint check | Code quality |
| npm run preview | Preview built app | Pre-deploy |

Branch & commit conventions:

1. Branch from main: git checkout -b feat/my-feature
2. Make changes in small, focused commits.
3. Write commit messages in conventional format: feat: add user dashboard, fix: resolve login redirect
4. Push branch and open PR against main.
5. Ensure CI passes (lint + build + tests).
6. Squash-merge into main with a clean commit message.

10.0

# Troubleshooting

Common failures a new developer encounters on day one:

| Symptom | Likely Cause | Fix |
|---|---|---|
| npm install fails with EACCES | Permission error on node_modules | Use nvm or reinstall Node.js via package manager |
| Dev server won't start — port in use | Another process on port 5173 / 3000 | Kill the process or set PORT env var |
| Blank page, no errors | Missing .env file or wrong variables | Copy .env.example to .env and fill values |
| Auth: login fails silently | Wrong Supabase URL or anon key | Verify VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY |
| Data not loading (console: 401/403) | Supabase RLS policy missing or wrong | Add appropriate SELECT policy for the table |
| TypeScript build errors | Type mismatch after dependency update | Run tsc --noEmit to see all errors at once |
| npm run build fails with out of memory | Node heap too small for large builds | Set NODE_OPTIONS=--max-old-space-size=4096 |
| Changes not reflecting in browser | Browser cache or HMR issue | Hard refresh (Cmd+Shift+R) or clear cache |

11.0

# Glossary

Project-specific terminology and abbreviations used throughout this manual:

| Term | Definition |
|---|---|
| RLS | Row-Level Security — PostgreSQL feature that restricts which rows a user can query based on their auth UID. |
| SPA | Single Page Application — a web app that loads one HTML page and dynamically updates content via JavaScript. |
| Supabase | Open-source Firebase alternative — provides PostgreSQL database, auth, and realtime subscriptions. |
| Vite | Next-generation build tool — fast dev server with HMR and optimized production builds via Rollup. |
| JWT | JSON Web Token — compact, URL-safe token format for representing claims between parties. Used for auth sessions. |